Course Code: 2165119 **Course Name:** ASD&D Lab

Class: TE **Batch:** T23 **Semester:** V

Student Name: Swaraj Kanse **Roll No:** 2511131

EXPERIMENT NO. 02

Aim: - To Containerize Application using Docker.

Theory:

Docker is an open-source platform that enables developers to build, deploy, run, update, and manage containers. It provides a standard way to package your application's code, configurations, and dependencies into a single object.

Core Docker Concepts:

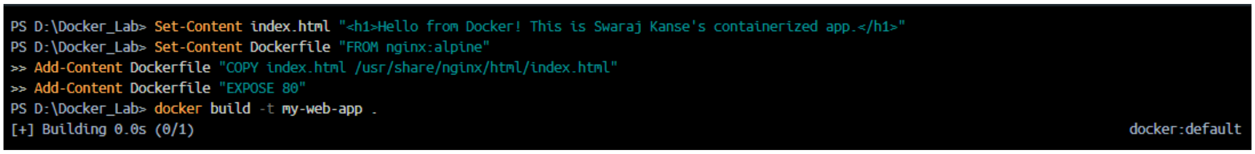
1. **Container:** A lightweight, standalone, and executable software package that includes everything needed to run an application (code, runtime, system tools, system libraries, and settings).
2. **Docker Image:** A read-only template that contains a set of instructions for creating a container that can run on the Docker platform.
3. **Dockerfile:** A simple text file that contains a list of commands that the Docker client calls while creating an image.
4. **Docker Engine:** The underlying client-server technology that builds and runs containers.

By containerizing an application, we ensure that it will run exactly the same regardless of the environment (development, testing, or production), solving the classic "it works on my machine" problem.

IMPLEMENTATION

Step 1: Setting up the environment and creating required files

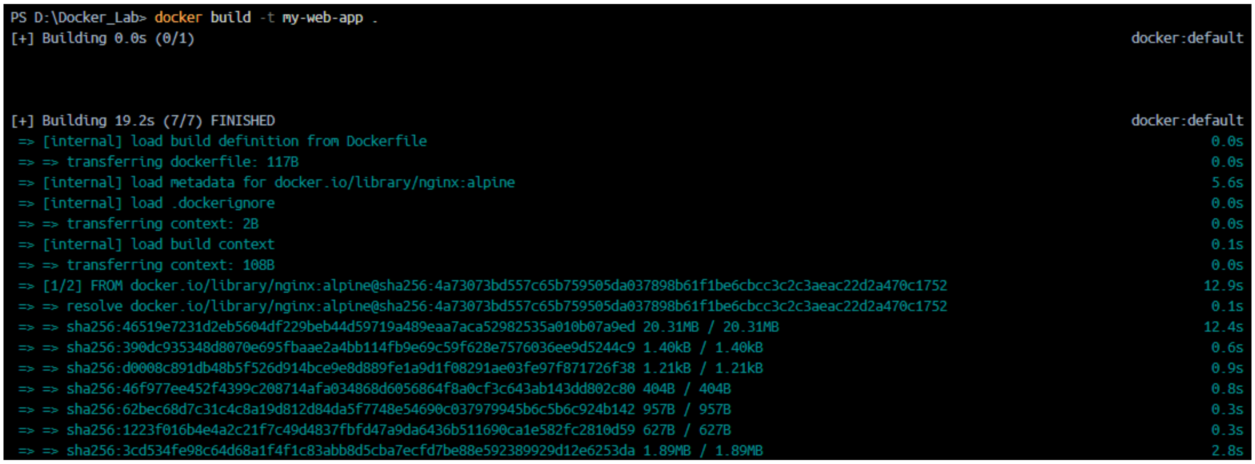
Creating a simple HTML file and a Dockerfile using PowerShell.



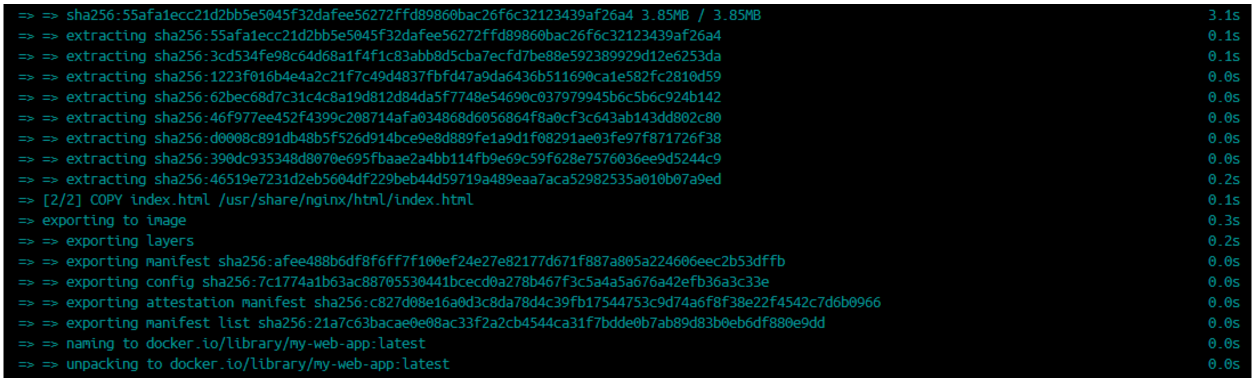
```
PS D:\Docker_Lab> Set-Content index.html "<h1>Hello from Docker! This is Swaraj Kanse's containerized app.</h1>"
PS D:\Docker_Lab> Set-Content Dockerfile "FROM nginx:alpine"
>> Add-Content Dockerfile "COPY index.html /usr/share/nginx/html/index.html"
>> Add-Content Dockerfile "EXPOSE 80"
PS D:\Docker_Lab> docker build -t my-web-app .
[+] Building 0.0s (0/1)                                                                          docker:default
```

Step 2: Building the Docker Image

Using the docker build command to create an image named 'my-web-app' from the current directory.



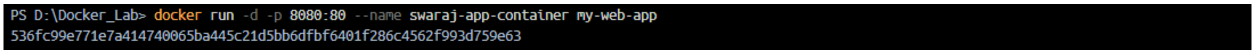
```
PS D:\Docker_Lab> docker build -t my-web-app .
[+] Building 0.0s (0/1)                                                                                                   docker:default



[+] Building 19.2s (7/7) FINISHED                                                                                         docker:default
 => [internal] load build definition from Dockerfile                                                                                0.0s
 => => transferring dockerfile: 117B                                                                                                0.0s
 => [internal] load metadata for docker.io/library/nginx:alpine                                                                     5.6s
 => [internal] load .dockerignore                                                                                                   0.0s
 => => transferring context: 2B                                                                                                     0.0s
 => [internal] load build context                                                                                                   0.1s
 => => transferring context: 108B                                                                                                   0.0s
 => [1/2] FROM docker.io/library/nginx:alpine@sha256:4a73073bd557c65b759505da037898b61f1be6cbcc3c2c3aeac22d2a470c1752              12.9s
 => => resolve docker.io/library/nginx:alpine@sha256:4a73073bd557c65b759505da037898b61f1be6cbcc3c2c3aeac22d2a470c1752               0.1s
 => => sha256:46519e7231d2eb5604df229beb44d59719a489eaa7aca52982535a010b07a9ed 20.31MB / 20.31MB                                   12.4s
 => => sha256:390dc935348d8070e695fbaae2a4bb114fb9e69c59f628e7576036ee9d5244c9 1.40kB / 1.40kB                                      0.6s
 => => sha256:d0008c891db48b5f526d914bce9e8d889fe1a9d1f08291ae03fe97f871726f38 1.21kB / 1.21kB                                      0.9s
 => => sha256:46f977ee452f4399c208714afa034868d6056864f8a0cf3c643ab143dd802c80 404B / 404B                                          0.8s
 => => sha256:62bec68d7c31c4c8a19d812d84da5f7748e54690c037979945b6c5b6c924b142 957B / 957B                                          0.3s
 => => sha256:1223f016b4e4a2c21f7c49d4837fbfd47a9da6436b511690ca1e582fc2810d59 627B / 627B                                          0.3s
 => => sha256:3cd534fe98c64d68a1f4f1c83abb8d5cba7ecfd7be88e592389929d12e6253da 1.89MB / 1.89MB                                      2.8s
```



```
 => => sha256:55afa1ecc21d2bb5e5045f32dafee56272ffd89860bac26f6c32123439af26a4 3.85MB / 3.85MB                                     3.1s
 => => extracting sha256:55afa1ecc21d2bb5e5045f32dafee56272ffd89860bac26f6c32123439af26a4                                          0.1s
 => => extracting sha256:3cd534fe98c64d68a1f4f1c83abb8d5cba7ecfd7be88e592389929d12e6253da                                          0.1s
 => => extracting sha256:1223f016b4e4a2c21f7c49d4837fbfd47a9da6436b511690ca1e582fc2810d59                                          0.0s
 => => extracting sha256:62bec68d7c31c4c8a19d812d84da5f7748e54690c037979945b6c5b6c924b142                                          0.0s
 => => extracting sha256:46f977ee452f4399c208714afa034868d6056864f8a0cf3c643ab143dd802c80                                          0.0s
 => => extracting sha256:d0008c891db48b5f526d914bce9e8d889fe1a9d1f08291ae03fe97f871726f38                                          0.0s
 => => extracting sha256:390dc935348d8070e695fbaae2a4bb114fb9e69c59f628e7576036ee9d5244c9                                          0.0s
 => => extracting sha256:46519e7231d2eb5604df229beb44d59719a489eaa7aca52982535a010b07a9ed                                          0.2s
 => [2/2] COPY index.html /usr/share/nginx/html/index.html                                                                         0.1s
 => exporting to image                                                                                                             0.3s
 => => exporting layers                                                                                                            0.2s
 => => exporting manifest sha256:afee488b6df8f6ff7f100ef24e27e82177d671f887a805a224606eec2b53dffb                                  0.0s
 => => exporting config sha256:7c1774a1b63ac88705530441bcecd0a278b467f3c5a4a5a676a42efb36a3c33e                                    0.0s
 => => exporting attestation manifest sha256:c827d08e16a0d3c8da78d4c39fb17544753c9d74a6f8f38e22f4542c7d6b0966                      0.0s
 => => exporting manifest list sha256:21a7c63bacae0e08ac33f2a2cb4544ca31f7bdde0b7ab89d83b0eb6df880e9dd                             0.0s
 => => naming to docker.io/library/my-web-app:latest                                                                               0.0s
 => => unpacking to docker.io/library/my-web-app:latest                                                                            0.0s
```

Step 3: Running the Container

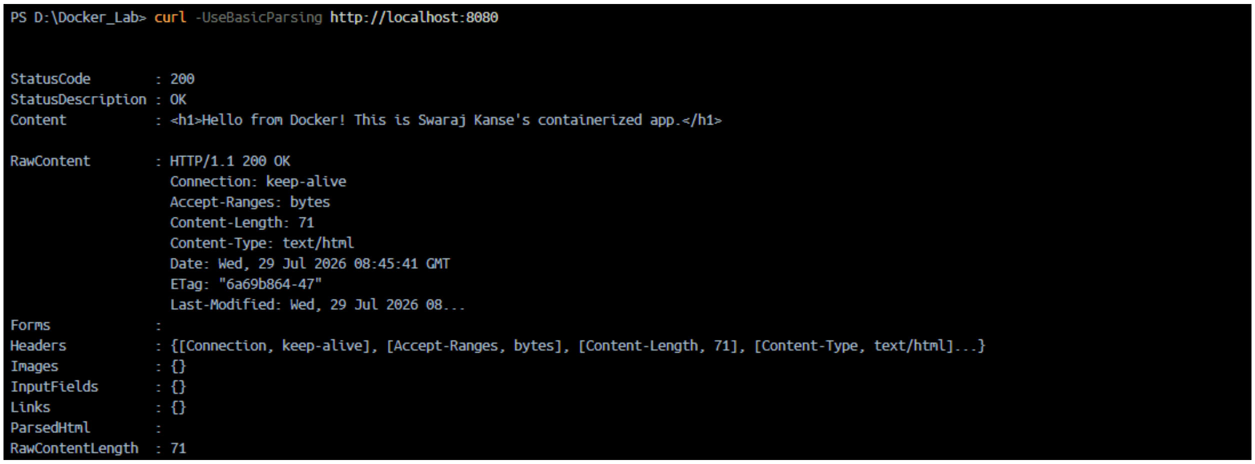
Using the docker run command to start the container, mapping host port 8080 to container port 80, and running it in detached mode.



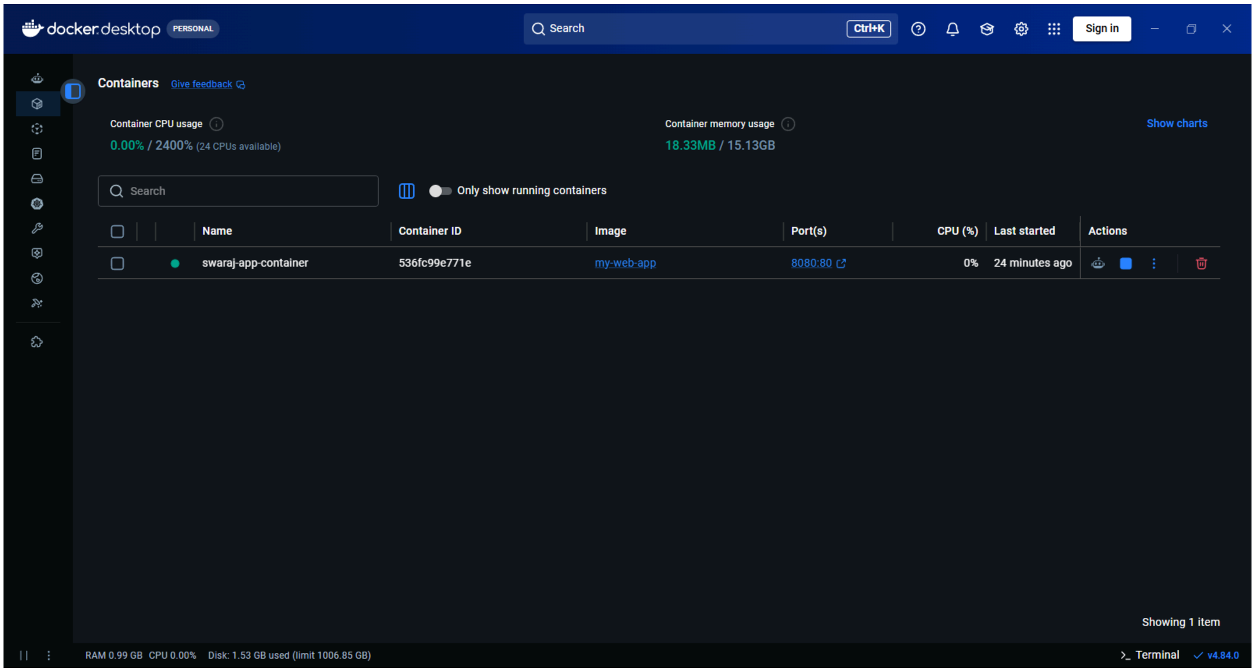
```
PS D:\Docker_Lab> docker run -d -p 8080:80 --name swaraj-app-container my-web-app
536fc99e771e7a414740065ba445c21d5bb6dfbf6401f286c4562f993d759e63
```

Step 4: Testing the Containerized Application

Verifying the web server is running and serving the custom HTML page.



```
PS D:\Docker_Lab> curl -UseBasicParsing http://localhost:8080


StatusCode        : 200
StatusDescription : OK
Content           : <h1>Hello from Docker! This is Swaraj Kanse's containerized app.</h1>

RawContent        : HTTP/1.1 200 OK
                    Connection: keep-alive
                    Accept-Ranges: bytes
                    Content-Length: 71
                    Content-Type: text/html
                    Date: Wed, 29 Jul 2026 08:45:41 GMT
                    ETag: "6a69b864-47"
                    Last-Modified: Wed, 29 Jul 2026 08...
Forms             :
Headers           : {[Connection, keep-alive], [Accept-Ranges, bytes], [Content-Length, 71], [Content-Type, text/html]...}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        :
RawContentLength  : 71
```

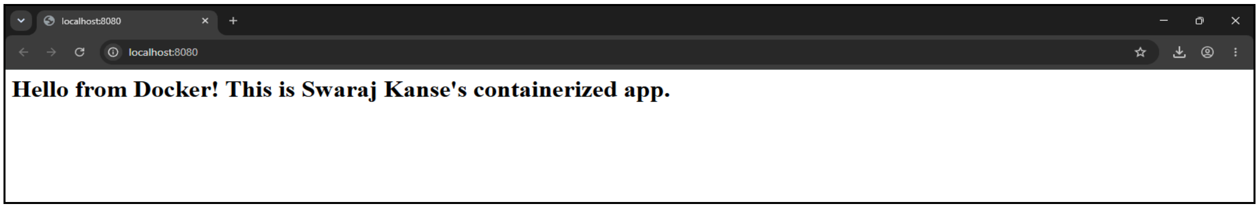


Container CPU usage
0.00% / 2400% (24 CPUs available)

Container memory usage
18.33MB / 15.13GB

☐		Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
☐	●	swaraj-app-container	536fc99e771e	my-web-app	8080:80	0%	24 minutes ago	

RAM 0.99 GB CPU 0.00% Disk: 1.53 GB used (limit 1006.85 GB)



Hello from Docker! This is Swaraj Kanse's containerized app.

Conclusion: Thus, we have successfully studied the concepts of containerization and used Docker to build an image from a Dockerfile, run it as an isolated container, and serve a web application via an Nginx server.